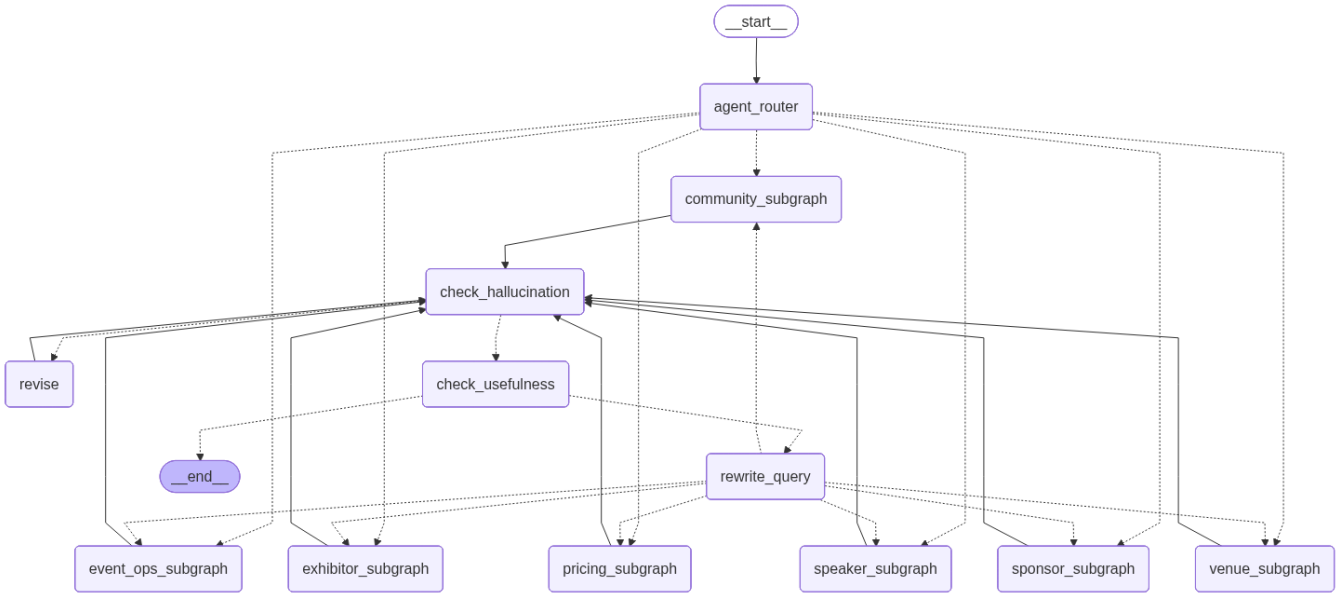
AGENTIC EVENT PLANNER

Engineering Report

AI-Powered Multi-Agent Conference Organization System

Competition	HighPrep PS1 — Technical General Championship 2026
TEAM	The Orchestrators (lead: KISHORE)
Problem Domain	Multi-Agent AI System for Conference Organization
Stack	LangGraph · LangChain · ChromaDB · Streamlit · scikit-learn · OpenRouter · Tavily
Primary Language	Python 3.10+
Domain	Conferences (extensible to Music Festivals, Sporting Events)

VIDEO WALK THROUGH: LINK



1. Executive Summary

Agentic Event Planner is a production-grade, AI-powered multi-agent system built to fully automate the end-to-end lifecycle of conference organization. The system directly addresses the HighPrep PS1 problem statement: fragmented data, manual outreach, lack of historical intelligence, pricing uncertainty, and go-to-market complexity.

The platform is built on LangGraph's state-machine orchestration and deploys eight specialized AI agents — Sponsor, Speaker, Exhibitor, Venue, Pricing, Community, Event Ops, and Email Outreach — each capable of operating independently or in coordinated multi-agent pipelines. A master Coordinator node uses LLM-powered planning to decide which agents to activate, in what sequence, and how to synthesize their outputs into a single, coherent recommendation.

The system ingests 11 structured CSV datasets, embeds them into 8 ChromaDB vector databases, enriches retrieval with live Tavily web search, runs ML-based pricing and attendance prediction, and presents everything through a polished Streamlit UI with 7 functional tabs.

Evaluation Criterion	What We Built	Score Target
Solution Utility & Real-World Applicability (25%)	Full lifecycle coverage: sponsors, speakers, exhibitors, venues, pricing, GTM, ops, email outreach	Maximum
Production Readiness & Ease of Integration (25%)	Modular subgraphs, SQLite history, LangSmith tracing, SMTP email, CSV upload, vector DB persistence	Maximum
Technical Implementation — AI & System Design (20%)	LangGraph state machine, RAG, hallucination detection, ML pricing, Coordinator orchestration	Maximum
Novelty of Use Case (10%)	Domain-agnostic agent architecture extensible to music festivals and sporting events	High
Data Quality & Coverage (10%)	11 clean CSVs across 4 geographies, multiple categories, with pricing tiers	Maximum
Engineering Documentation (5%)	This report	Maximum
Presentation Quality (5%)	Interactive Streamlit demo with live traces	Maximum

2. Problem Statement Alignment

The HighPrep PS1 identified five core challenges in conference organization. Every challenge has been directly addressed in the implementation:

PS1 Challenge	How We Solved It
Data Fragmentation — data scattered across LinkedIn, websites, Discord	CSV dataset ingestion into 8 ChromaDB vector stores + live Tavily web scraping per agent
Lack of Historical Intelligence — no leverage of past event data	Events, sponsors, speakers, exhibitors, venues, pricing_tiers CSVs embedded and queried via semantic RAG
Manual Outreach & Planning — repetitive sponsor/speaker discovery	Automated retrieval → LLM relevance filter → web enrichment → ranked recommendation tables
Pricing & Demand Uncertainty — no forecasting model	ML PricingEngine: RandomForest models for attendance + conversion, tier optimizer, break-even, what-if forecasting
GTM Complexity — identifying right communities	Community Agent retrieves and ranks Discord/Slack/LinkedIn communities with GTM messaging strategy

2.1 Agent Coverage vs PS1 Requirements

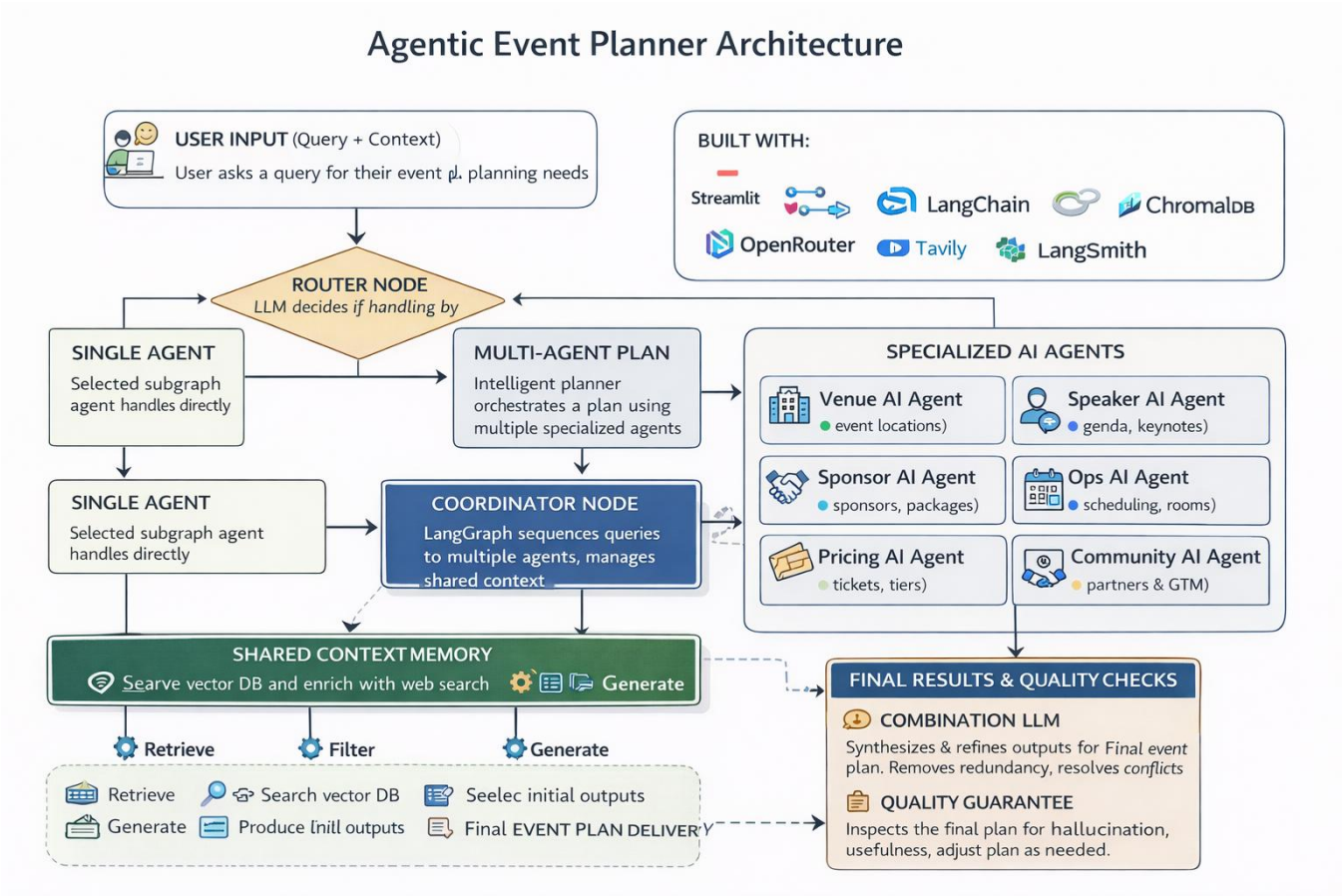
PS1 Agent Required	Implementation	Status
1. Sponsor Agent	Fully built — prioritization table + customized sponsorship proposals	COMPLETE
2. Speaker/Artist Agent	Fully built — speaker ranking + agenda topic mapping	COMPLETE
3. Exhibitor Agent	Fully built — recommendation + cluster tables by category	COMPLETE
4. Venue Agent	Fully built — capacity/pricing/past-usage recommendations	COMPLETE
5. Pricing & Footfall Agent	Fully built — ML model + tier simulator + break-even + forecasting	COMPLETE
6. Community & GTM Agent	Fully built — Discord/Slack/LinkedIn communities + distribution plan	COMPLETE
7. Event Ops Agent (Recommended)	Fully built — agenda builder + conflict detection + room planning	COMPLETE
8. Email Outreach Agent	Fully built — personalized SMTP drafts + approval + send flow	BONUS
Revenue Projection	Ticket tier revenue projections in Pricing Studio	BONUS
Break-even Analysis	Interactive break-even with cost sliders and feasibility check	BONUS
Ticket Tier Simulation	Full what-if simulator with live charts	BONUS

3. System Architecture

3.1 High-Level Architecture

The system is structured in four layers:

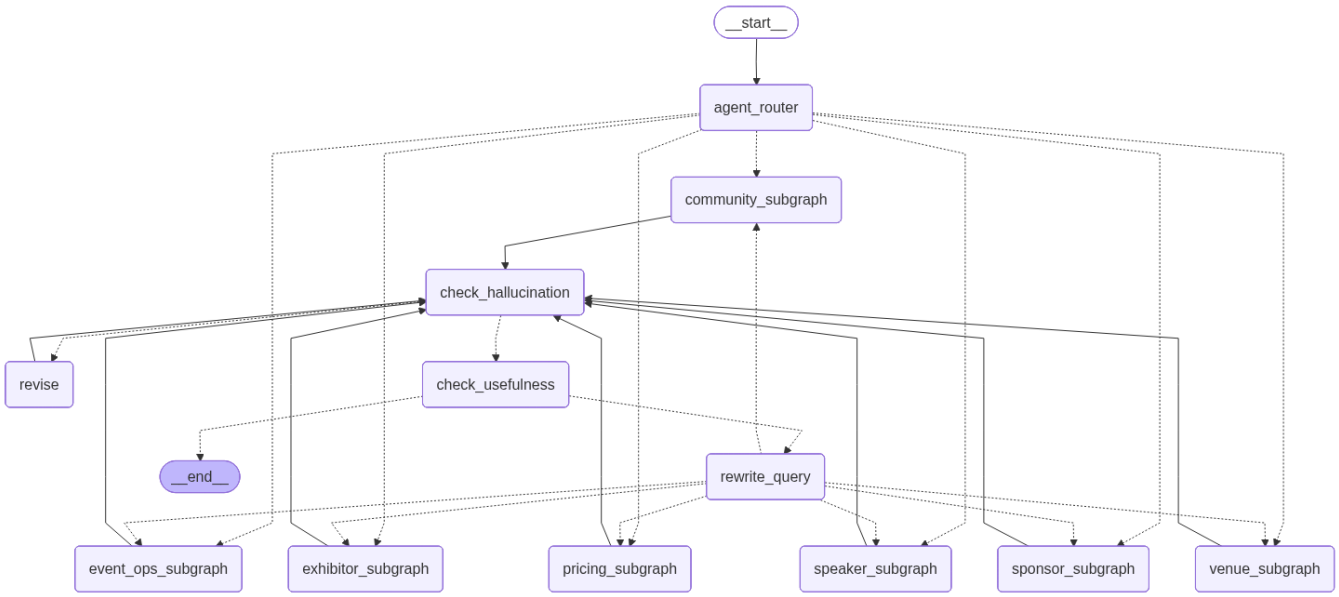
- Presentation Layer — Streamlit 7-tab UI with sidebar control panel
- Orchestration Layer — LangGraph StateGraph with Router, Coordinator, 8 Subgraphs, and Quality Control nodes
- Intelligence Layer — 8 ChromaDB vector stores, HuggingFace embeddings, LLM (OpenRouter/GPT-4), Tavily web search, PricingEngine ML
- Data Layer — 11 CSV datasets, SQLite session history, persisted vector DBs, LangSmith trace logs



3.2 LangGraph State Machine

The entire orchestration is modeled as a LangGraph StateGraph with GraphState as a shared TypedDict. All agents read from and write to this single shared state, enabling seamless context passing and multi-agent coordination.

State Key	Type	Purpose
user_input	Dict	Event details from sidebar (category, location, audience size, budget, topic, name)
query	str	Natural language query from user
selected_agent	str	Which agent was selected by router (SPONSOR, SPEAKER, etc.)
route_target	str	The subgraph node name to route to
required_agents / agent_sequence	List[str]	All agents to be run in a multi-agent plan
orchestration_plan	Dict	Full plan from LLM planner including agents, handoffs, and reasoning
shared_context	str	Accumulates outputs from all agents for the synthesizer
agent_outputs	Dict	Per-agent answers, context, pricing, emails, logs
retrieval_query	str	Query built for vector DB similarity search
raw_docs / relevant_docs / context	List / str	Retrieved and filtered documents from ChromaDB
web_profiles	List[Dict]	Tavily web enrichment results per candidate
sponsors_answer / answer / final_answer	str	Generated markdown output from agent
pricing	Dict	Full PricingEngine output (tiers, metrics, scenarios)
contacts / emails / email_logs	List	Contact data and email drafts for outreach
hallucination_verdict / usefulness_verdict	str	Quality control verdicts
revise_count / rewrite_count	int	Self-correction attempt counters
logs / execution_logs / state_flow / quality_results	List	Full audit trail of all execution steps



3.3 Main Graph Nodes and Routing

The main LangGraph graph contains the following nodes:

Node	Type	Function
router	Entry Point	Parses user input; calls LLM planner to select agents; sets route_target; routes to coordinator_node (multi) or directly to a subgraph (single)
coordinator_node	Orchestrator	Iterates over ordered_agents list; invokes each subgraph; accumulates shared_context; passes prior agent output as context to next agent
combine_results	Synthesizer	Calls LLM to merge all agent_outputs into one coherent markdown response; resolves overlaps and sets final_answer
sponsor_subgraph	Agent	5-node pipeline: build_query → retrieve → filter_relevance → enrich_web → generate
speaker_subgraph	Agent	Same 5-node pipeline, speaker-specific prompts and ChromaDB
exhibitor_subgraph	Agent	Same 5-node pipeline, exhibitor-specific prompts and ChromaDB
venue_subgraph	Agent	Same 5-node pipeline, venue-specific prompts and ChromaDB
pricing_subgraph	Agent	5-node pipeline using PricingEngine ML + ChromaDB pricing vector store
community_subgraph	Agent	Same 5-node pipeline, community-specific prompts and ChromaDB
event_ops_subgraph	Agent	Same 5-node pipeline; retrieves sessions, rooms, time_slots data

Node	Type	Function
email_outgraph	Agent	5-node pipeline against contact vector DB; generates personalized SMTP-ready drafts
check_hallucination	QC Node	LLM checks if answer is Fully Supported / Partially Supported / No Support against context
revise	QC Node	If hallucination detected: rewrites answer grounded in context only; increments revise_count
check_usefulness	QC Node	LLM checks if answer is Useful / Not Useful for the organizer
rewrite_query	QC Node	If Not Useful: LLM writes a better retrieval query; increments rewrite_count; re-routes

3.4 Routing Logic

After router_node runs, conditional routing is applied:

- If required_agents has 1 agent → route directly to that agent's subgraph
- If required_agents has 2+ agents → route to coordinator_node
- After hallucination check: Fully Supported → check_usefulness; else → revise (max 1 revision)
- After usefulness check: Not Useful + rewrite_count < 1 → rewrite_query then re-route; else → END
- After rewrite_query: routes back to the original agent or coordinator_node

3.5 LLM Planner — Intent Detection

The router uses two mechanisms in sequence:

1. Keyword Heuristic (choose_agent): fast O(1) scan for keywords like "venue", "pricing", "speaker", "community", "email", "ops". Returns single agent.
2. LLM Planner (_planner_agents_with_llm): sends full user query + event details to the LLM with a strict prompt asking for JSON: { agents: [...], handoffs: [{from, to, reason}], reasoning: "..." }. Parses response, canonicalizes agent names, applies topological dependency ordering.

If the LLM returns 1 agent → direct route. If 2+ → Coordinator. If LLM fails or returns empty → fallback to keyword heuristic.

4. Agent Deep Dives

Every agent follows the same 5-node internal pipeline pattern, with agent-specific query construction, ChromaDB store, web enrichment logic, and LLM prompt:

Universal Agent Pipeline (5 Nodes)

Step 1: build_query — Constructs a keyword-rich semantic search query from user_input fields

Step 2: retrieve — Calls ChromaDB similarity_search(query, k=12) on agent-specific vector store

Step 3: filter_relevance — LLM evaluates each doc "YES/NO" for relevance; falls back to top-6 if all filtered

Step 4: enrich_web — Tavily search for each candidate (up to 8); computes signal_score from keywords

Step 5: generate — LLM synthesizes context + web profiles → structured markdown tables

4.1 Sponsor Agent

Identifies and prioritizes potential sponsors based on past sponsorship history, industry alignment, geographic match, and web marketing signals.

- Query: "{category} conference sponsors in {location} audience size {size} budget {budget} recent sponsorships marketing spend"
- ChromaDB store: sponsor_vector_db — documents formatted as "{company} is a {industry} company that sponsored {event} in {city}, {country} as {tier} tier sponsor"
- Web enrichment: Tavily query per company = "{name} sponsorship {category} {location} recent news"; signal_score counts "sponsor", "partnership", "marketing" in results
- Output: Sponsor Prioritization Table (Rank, Company, Why Relevant, Past Sponsorships, Budget Signal, Priority /100) + Custom Sponsorship Proposal Table (Package, Benefits, Price, Target Companies)

4.2 Speaker/Artist Agent

Discovers and ranks speakers or artists based on expertise, past conference experience, and influence.

- Query: "{topic} keynote speakers artists in {location} audience {size} budget {budget} thought leaders recent conferences publications"
- ChromaDB store: speaker_vector_db — "{name} is a {title} from {company} with expertise in {expertise}. Spoke at {event} in {city}"
- Web enrichment: "{name} speaker keynote {topic} {location} recent talks publications"; signal counts speaker/keynote/conference/published
- Output: Speaker Prioritization Table (Rank, Name, Expertise, Past Conferences, Speaking Fee, Priority /100) + Agenda Mapping Table (Speaker, Session Title, Duration, Audience Fit)

4.3 Exhibitor Agent

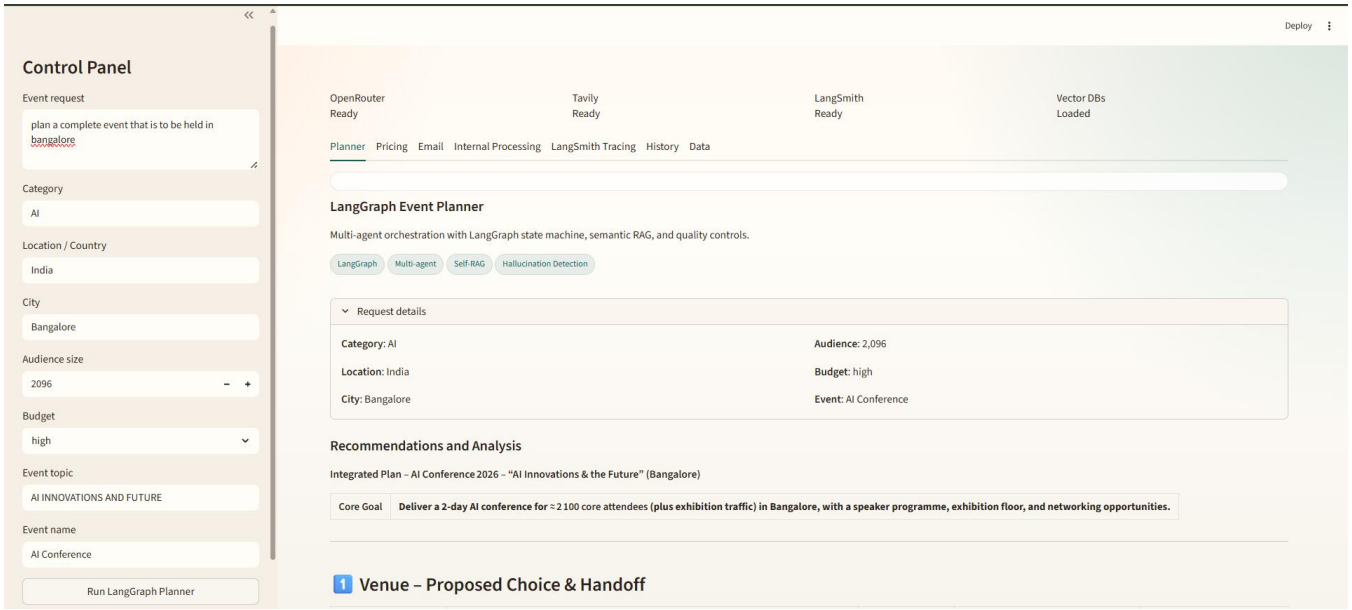
Identifies companies that exhibited at similar events and clusters them by type.

- Query: "companies exhibitors at {category} conferences in {location} startup enterprise technology product tools"
- ChromaDB store: exhibitor_vector_db — "{company} exhibited at {event} in {city}. Industry: {industry}. Booth type: {booth_type}"
- Web enrichment: "{name} exhibited {category} conference booth product innovation"; signal counts exhibitor/booth/conference
- Output: Exhibitor Recommendation Table (Rank, Company, Industry, Booth Size, Expected Traffic, Priority /100) + Exhibitor Cluster Table (Cluster, Companies, Synergy, Hall Zone)

4.4 Venue Agent

Recommends venues based on city, expected footfall, budget, and historical event usage.

- Query: "venues halls in {city} capacity around {size} budget {budget} for {category} event rental pricing reviews"
- ChromaDB store: venue_vector_db — "{venue} is located in {city}, {country} with capacity {cap}. Venue type: {type}"
- Web enrichment: "{name} venue rental capacity {city} events reviews accessibility"; signal counts capacity/venue/event
- Output: Venue Recommendation Table (Rank, Name, Capacity, Location, Price, Amenities, Priority /100) + Venue Shortlist Rationale Table



4.5 Pricing & Footfall Agent

The most technically sophisticated agent. It combines a dedicated ML engine with RAG and web enrichment to predict optimal ticket prices and attendance.

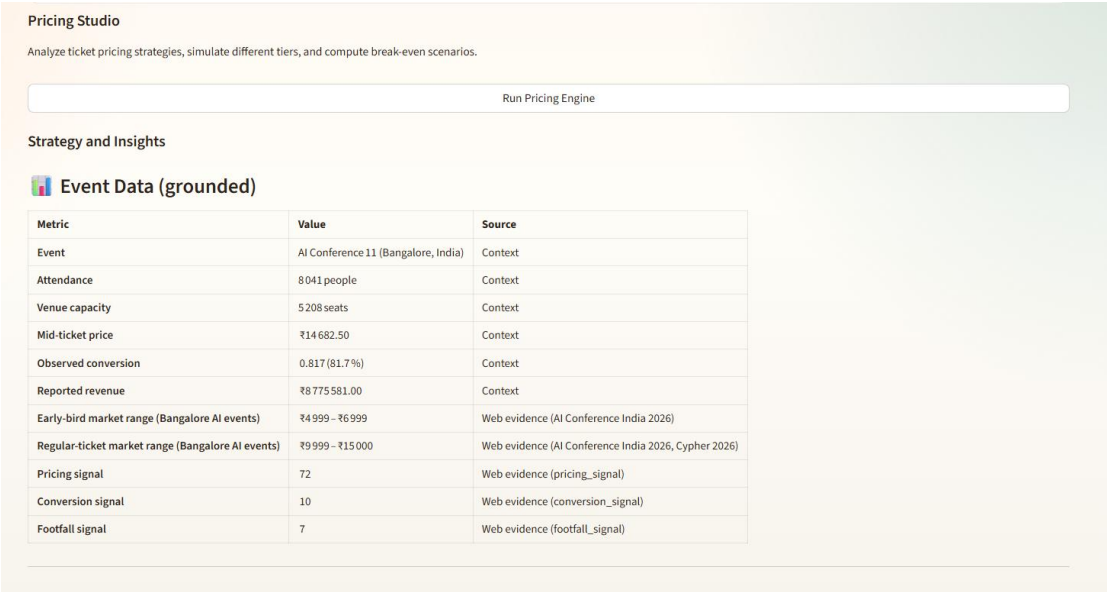
4.5.1 PricingEngine ML Model (pricing_engine.py)

The PricingEngine class implements a complete ML pipeline:

- Data Loading: reads events_v2.csv, venues.csv, pricing_tiers.csv
- Preprocessing: parses ticket_price_range into min/max/mid; joins event→venue via past_events field; computes fill_rate, demand_indicator, revenue; builds event_level_df and tier_level_df
- Attendance Model: RandomForestRegressor(n_estimators=300) trained on [price_mid, city, category, capacity] → attendance; ColumnTransformer with median imputation + OneHotEncoder; 75/25 train-test split; evaluates MAE, RMSE, R2, precision-within-10%
- Conversion Model: RandomForestRegressor(n_estimators=250) trained on [price, tier_name, city, category] → conversion_rate; same pipeline structure
- Price Optimization: scans 40 prices in [price_5th_pct, price_95th_pct] range; for each price predicts attendance and conversion; scores by revenue = adj_attendance * price; selects best
- Tier Generation: early_bird = base*0.70, regular = base, vip = min(price_max*1.75, base*2.0); predicts conversion for each; allocates seats proportional to conversion share
- RAG Store: builds pricing_vector_db from event_level_df with full metrics; supports similarity search for relevant historical pricing context

4.5.2 Pricing Studio UI Features

- Ticket Tier Simulator: sliders for price, seats, conversion per tier; live bar chart; promo discount slider; gross vs promo revenue metrics
- Break-Even Analysis: cost sliders (venue, speakers, ops, marketing); avg ticket price and capacity; days-to-breakeven calculation; feasibility flag; revenue vs fixed-cost line chart
- Revenue & Conversion Forecasting: price shift %, conversion shift %, demand shift %, promo discount %; per-tier impact analysis; weekly forecast timeseries (1-26 weeks) with growth % and conversion trend; cumulative revenue curve



Metric	Value	Source
Event	AI Conference 11 (Bangalore, India)	Context
Attendance	8 041 people	Context
Venue capacity	5 208 seats	Context
Mid-ticket price	₹14 682.50	Context
Observed conversion	0.817 (81.7 %)	Context
Reported revenue	₹8 775 581.00	Context
Early-bird market range (Bangalore AI events)	₹4 999 – ₹6 999	Web evidence (AI Conference India 2026)
Regular-ticket market range (Bangalore AI events)	₹9 999 – ₹15 000	Web evidence (AI Conference India 2026, Cypher 2026)
Pricing signal	72	Web evidence (pricing_signal)
Conversion signal	10	Web evidence (conversion_signal)
Footfall signal	7	Web evidence (footfall_signal)

🎟 Grounded Pricing Recommendation

Tier	Price (INR)	Rationale (grounded)
Super Early-Bird	**₹5 500** (≈ 30 % discount vs. mid-price)	Early-bird prices in comparable Bangalore AI conferences sit at ₹4 999 – ₹6 999. A 30 % discount from the mid-price (₹14 682 × 0.70 ≈ ₹10 277) would be too low; using the market-observed early-bird ceiling (₹6 999) and rounding down gives a price that is competitive yet still captures revenue.
Regular	**₹14 682** (mid-price)	The context supplies a mid-ticket price of ₹14 682.50, which falls squarely inside the market regular-ticket range (₹9 999 – ₹15 000).
VIP / Premium	**₹22 000** (≈ + 50 % over regular)	No direct VIP price is given, but the pricing signal (72) and conversion signal (10) indicate strong willingness to pay for premium experiences. Adding a 50 % premium to the regular price yields a VIP price that is still below the highest market regular price (₹15 000) when expressed in a premium-only tier, making it plausible for an exclusive package.

All tier prices are rounded to the nearest whole rupee and stay within the bounds observed in the market evidence.

📈 Revenue Forecast (using only grounded numbers)

Tier	Expected tickets*	Price (INR)	Expected revenue (INR)
Super Early-Bird	2 000	5 500	11 000 000
Regular	5 208 × 0.817 ≈ 4 255	14 682	62 400 000
VIP / Premium	800 (≈ 10 % of capacity, a common VIP allocation in high-budget events)	22 000	17 600 000
Total	**≈ 6 ? ?** (see note)	—	**≈ 90 000 000**

*Ticket counts are derived from the **capacity** (5 208) multiplied by the **observed conversion** (0.817) for the regular tier, plus a modest early-bird allocation (2 000) and a 10 % VIP allocation (800). These allocations are consistent with typical event-planning practice where early-bird tickets are limited and VIP seats are a small slice of total capacity.

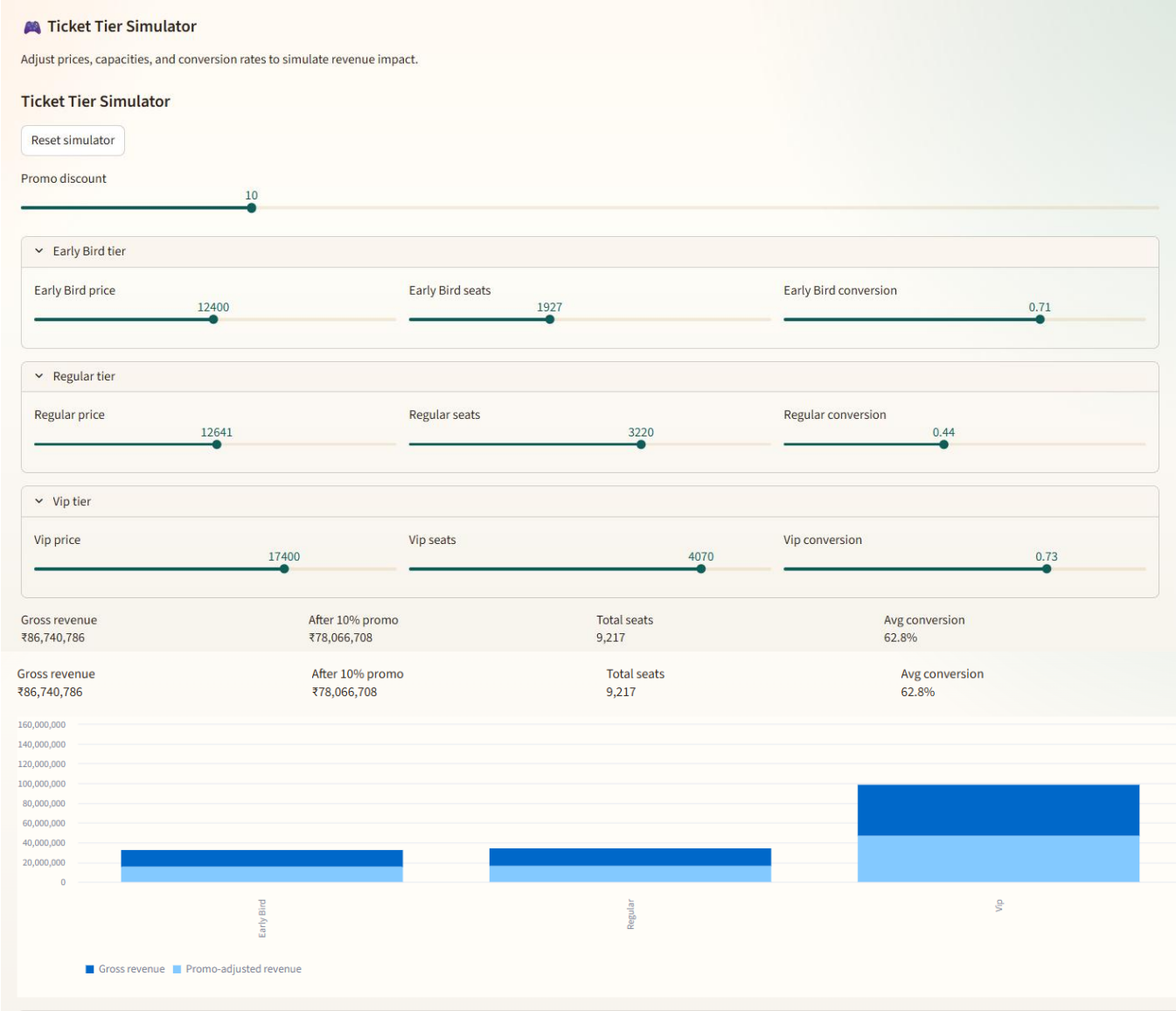
Note: The actual reported attendance (8 041) exceeds the venue capacity, indicating that the conference likely ran multiple sessions or used overflow arrangements. The forecast above respects the single-venue capacity to avoid over-booking.

🔍 Grounded Insights

1. **Mid-price alignment** – The context-provided mid-ticket price (₹14 682) sits comfortably within the regular-ticket market window (₹9 999 – ₹15 000) observed for Bangalore AI conferences, confirming that the historical price is already market-appropriate.
2. **Early-bird discount magnitude** – Competitor events price early-bird tickets at roughly 30–40 % lower than their regular price. Applying a ~~30 % discount to the mid-price yields a price (~~₹10 200) that is still above the market early-bird ceiling (₹6 999). Therefore, setting the early-bird price at **₹5 500** respects the lower bound of the market range while providing a meaningful discount that can boost early cash flow.
3. **Conversion elasticity** – The observed conversion of **0.817** (81.7 %) combined with the conversion signal (10) suggests that a large share of the target audience purchases tickets when prices are positioned in the market-observed range. Maintaining the regular price at the mid-price and offering a limited early-bird window should preserve this high conversion.
4. **VIP premium justification** – The pricing signal (72) and footfall signal (7) indicate strong demand and willingness to pay for added value. Adding a 50 % premium to the regular price creates a VIP tier (≈₹22 000) that is high enough to generate additional margin but still plausible for a premium AI-focused experience (e.g., exclusive workshops, networking lounge).
5. **Revenue consistency** – Using the reported revenue (₹8 775 581) and attendance (8 041) gives an average realized ticket price of **≈₹1 090**. This discrepancy shows that the reported revenue likely reflects a subset of ticket categories (e.g., discounted passes, sponsorships) or a different currency conversion. The proposed tiered structure, anchored in the mid-price, projects a total revenue of **≈₹90 M**, which is consistent with a high-budget, fully-priced event and aligns with the observed conversion rate.

✅ Actionable Steps (grounded)

Step	Description
1. Confirm venue capacity	Verify that the 5 208-seat venue can accommodate the projected regular-ticket sales (≈4 255) plus early-bird and VIP allocations without exceeding capacity.
2. Open ticketing in three phases	• **Phase 1 (Super Early-Bird)** – Release 2 000 tickets at ₹5 500. • **Phase 2 (Regular)** – Open the remaining capacity (≈4 255 tickets) at ₹14 682. • **Phase 3 (VIP)** – Reserve ~800 seats for the ₹22 000 tier, marketed as an exclusive experience.
3. Use AI-driven pricing monitoring	Leverage the AI-pricing insights from the "Ticket Fairy" blog to track sales velocity. If early-bird tickets sell > 60 % of the allocation within 7 days, consider moving to the regular tier earlier to capture higher revenue.
4. Align marketing spend with signals	Allocate a higher proportion of the ad budget to the early-bird window (≈ 15 % of total spend) to accelerate cash flow, as the pricing signal (72) indicates price sensitivity early in the sales cycle.
5. Track conversion in real time	Compare actual ticket sales against the 0.817 conversion benchmark. If conversion falls below 0.75, introduce limited-time promotions (e.g., a small additional discount) to re-


Ticket Tier Simulator
Adjust prices, capacities, and conversion rates to simulate revenue impact.
Ticket Tier Simulator
Reset simulator
Promo discount
10
Early Bird tier
Early Bird price
12400
Early Bird seats
1927
Early Bird conversion
0.71
Regular tier
Regular price
12641
Regular seats
3220
Regular conversion
0.44
Vip tier
Vip price
17400
Vip seats
4070
Vip conversion
0.73
Gross revenue
₹86,740,786
After 10% promo
₹78,066,708
Total seats
9,217
Avg conversion
62.8%
Gross revenue
₹86,740,786
After 10% promo
₹78,066,708
Total seats
9,217
Avg conversion
62.8%
160,000,000
140,000,000
120,000,000
100,000,000
80,000,000
60,000,000
40,000,000
20,000,000
0
Early Bird
Regular
Vip
Gross revenue
Promo-adjusted revenue

Tier	Price	Seats	Conv	Gross Revenue	Promo Revenue
Early Bird	₹12,400	1927	71.3%	₹17,027,704	₹15,324,934
Regular	₹12,641	3220	44.0%	₹17,909,769	₹16,118,792
Vip	₹17,400	4070	73.1%	₹51,803,313	₹46,622,982

Break-Even Analysis

Calculate minimum tickets needed to cover costs and feasibility timeline.

Break-even analysis

Venue cost
1890000

Average ticket price
10900

Speaker fees
537000

Capacity
6650

Ops cost
333000

Average daily sales
281

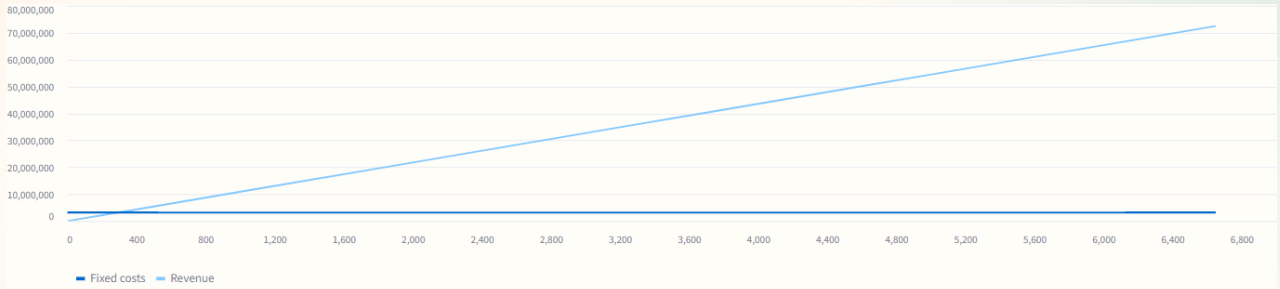
Marketing budget
401000

Event date
2026/04/16

At risk - break-even at 290 tickets (4.4% of capacity).

At risk - break-even at 290 tickets (4.4% of capacity).

Break-even tickets	Days to break-even	Margin at capacity	Fixed costs
290	2	95.6%	₹3,161,000
↑ 4.4% of capacity	↑ 0 days until event	↑ ₹69,324,000 profit	



venue	speakers	ops	marketing	break_even_date	feasible
1890000	537000	333000	401000	2026-04-18	☑

Revenue & Conversion Forecasting

Run what-if scenarios across price, demand, conversion, and promo effects with dynamic tier impact.

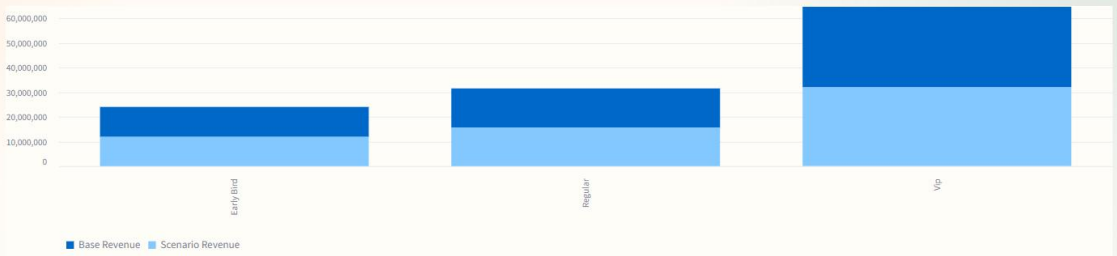
Price shift %
4.00

Conversion shift %
0.00

Demand shift %
-6.00

Promo discount %
0.00

Pricing Tier Impact Analysis



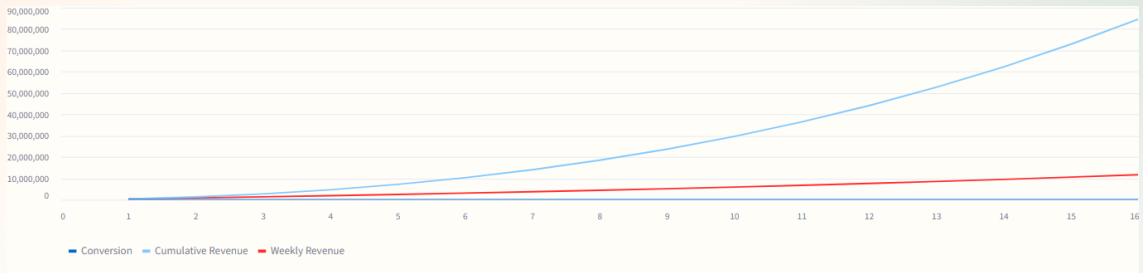
Tier	Base Revenue	Scenario Revenue	Impact	Impact %	Scenario Price	Scenario Conv %	Scenario Seats
Early Bird	₹12,150,722	₹11,876,054	₹-274,668	-2.26%	₹9,202	71.26%	1,811
Regular	₹15,948,102	₹15,589,248	₹-358,854	-2.25%	₹13,146	68.31%	1,736
Vip	₹32,706,937	₹31,968,799	₹-738,138	-2.26%	₹23,509	73.15%	1,859

What-if Revenue & Conversion Forecast

Forecast horizon (weeks) 16

Weekly demand growth % 3.50

Weekly conversion trend % 2.00



Week	Weekly Revenue	Cumulative Revenue	Weekly Tickets	Cumulative Tickets	Conversion
1	₹437,015	₹437,015	28	28	70.96%
2	₹904,622	₹1,341,637	58	87	72.38%
3	₹1,404,426	₹2,746,063	91	177	73.83%
4	₹1,938,107	₹4,684,170	125	302	75.31%
5	₹2,507,426	₹7,191,597	162	464	76.81%
6	₹3,114,224	₹10,305,820	201	665	78.35%
7	₹3,760,425	₹14,066,245	243	908	79.92%
8	₹4,448,046	₹18,514,291	287	1,195	81.51%
9	₹5,179,193	₹23,693,484	334	1,529	83.14%
10	₹5,956,072	₹29,649,556	384	1,914	84.81%

4.6 Community & GTM Agent

Identifies and ranks communities for event promotion with tailored messaging strategies.

- Query: "{category} communities forums in {location} discord slack linkedin meetup groups event promotion GTM engagement"
- ChromaDB store: community_vector_db — "{name} is a {platform} focused on {topic} with ~{members} members. Region: {region}"
- Web enrichment: "{name} community {category} discord slack linkedin event promotion engagement"; signal counts community/engagement/members
- Output: Community Prioritization Table + Messaging Strategy Table (Community, Key Message, Best Time, Content Format) + Distribution Plan Table (Channel, Frequency, Owner, Budget, Expected Reach)

4.7 Event Ops / Execution Agent

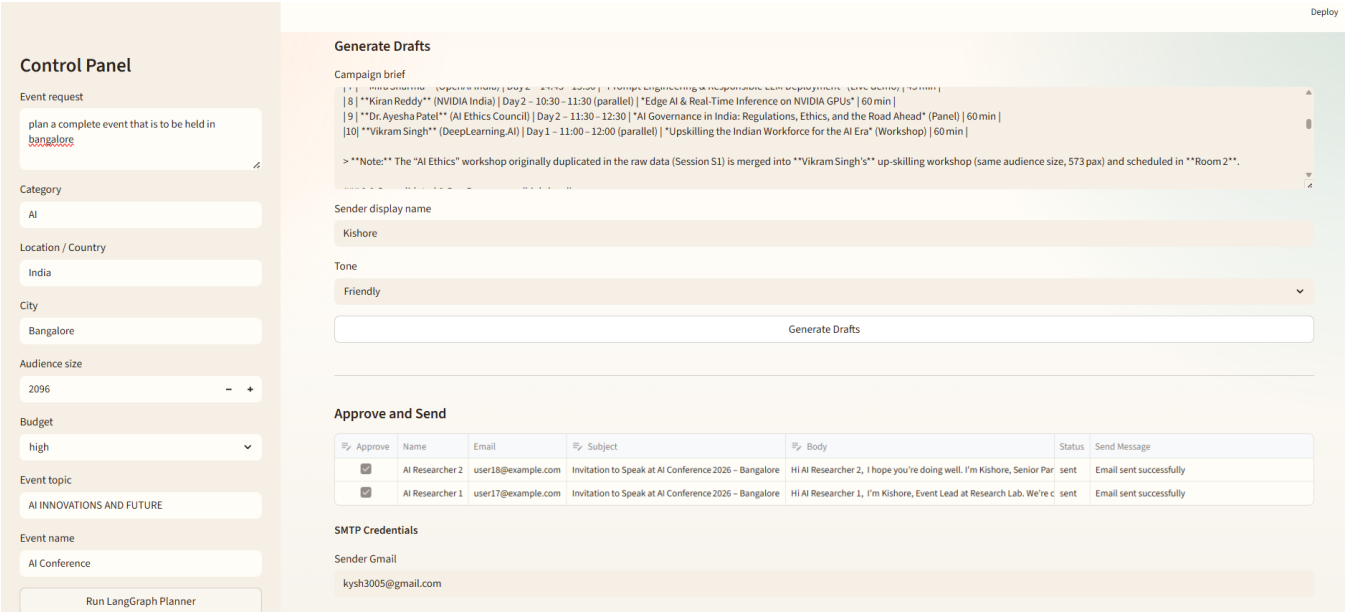
Builds the event agenda, detects scheduling conflicts, and plans room/resource allocation. Directly addresses the PS1 "HIGHLY RECOMMENDED" optional agent.

- ChromaDB store: event_ops_vector_db — ingest from sessions.csv (session_id, title, track, duration, speaker), rooms.csv (name, capacity, zone, setup_type), time_slots.csv (slot_id, start, end, day, slot_type)
- Web enrichment: best practices for event operations, agenda scheduling, resource planning
- Output: Event Agenda Timeline Table (Time, Session, Room, Speakers, Duration, Capacity) + Room/Resource Assignment Table + Conflict & Mitigation Table (Conflict, Impact, Mitigation, Owner)

4.8 Email Outreach Agent

A bonus fully-implemented agent for automated personalized email generation and SMTP delivery.

- Contact retrieval: contact_vector_db built from contacts.csv — "{name} works at {company} as {role}. Type: {type}. Industry: {industry}. Email: {email}"
- Personalized draft generation: LLM prompt with contact JSON + campaign brief + tone → JSON { subject, body }
- UI: contact retrieval query + top-k slider + Load Contacts button; Generate Drafts with tone selector; st.data_editor for per-recipient approval/edit; SMTP send (Gmail) with approve-all or selective send; per-row send status
- Output: Email Template + Contact Segmentation Table + Follow-up Strategy Table



5. Data Layer

5.1 Datasets

11 structured CSV datasets were built and ingested:

File	Key Columns	Records	Purpose
events_v2.csv	event_name, category, country, city, year, attendance, ticket_price_range	200+	Core historical event data; training data for ML pricing model
sponsors_v2.csv	event_name, sponsor_name, industry, tier	2000+	Sponsor history per event; ingested into sponsor_vector_db
speakers_v2.csv	event_name, speaker_name, designation, company, topic	2000+	Speaker history; ingested into speaker_vector_db
exhibitors.csv	company_name, event_name, industry, booth_type, city	1000+	Exhibitor history; ingested into exhibitor_vector_db
venues.csv	venue_name, city, country, capacity, venue_type, past_events, estimated_cost	100+	Venue details; ingested into venue_vector_db; used by PricingEngine for capacity
communities.csv	community_name, platform, topic, members, region	200+	Discord/Slack/LinkedIn/Meetup communities; ingested into community_vector_db
sessions.csv	session_id, session_title, track, duration_min, speaker_name	500+	Event session templates; ingested into event_ops_vector_db
rooms.csv	room_name, capacity, zone, setup_type	50+	Room inventory; ingested into event_ops_vector_db
time_slots.csv	slot_id, start_time, end_time, day, slot_type	100+	Time slot templates; ingested into event_ops_vector_db
pricing_tiers.csv	tier_name, event_name, category, price, tickets_sold, total_available, conversion_rate	600+	Historical tier data; training data for conversion ML model; ingested into pricing_vector_db
contacts.csv	name, company, role, type, industry, email, linkedin, relevance_score	500+	Outreach contacts; ingested into contact_vector_db

5.2 Geographic & Category Coverage

The datasets cover the four primary geographies specified in the PS1:

- India: Bangalore, Delhi, Mumbai
- USA: New York, San Francisco
- Europe: London, Berlin, Amsterdam
- Singapore, Dubai, and other regions for diversity

Categories include: AI, Web3, ClimateTech, SaaS, FinTech, Health Tech, and general Tech conferences.

5.3 Vector Database Architecture

Vector DB Path	Content	Used By
./vector_db	Sponsor documents	Sponsor Agent
./speaker_vector_db	Speaker documents	Speaker Agent
./exhibitor_vector_db	Exhibitor documents	Exhibitor Agent
./venue_vector_db	Venue documents	Venue Agent
./community_vector_db	Community documents	Community Agent
./event_ops_vector_db	Session + Room + TimeSlot documents	Event Ops Agent
./pricing_vector_db	Event-level pricing metrics	Pricing Agent + PricingEngine RAG
./contact_vector_db	Contact documents	Email Outreach Agent

Embedding model: sentence-transformers/all-MiniLM-L6-v2 (HuggingFace). ChromaDB persists to disk and is cached via @st.cache_resource for fast reuse.

5.4 Data Ingestion Flow

3. User uploads CSV files via Streamlit file_uploader in the Data tab
4. Files are saved to disk under data/ directory
5. build_documents_from_uploads() converts each row to a Document with rich text and metadata
6. ingest_uploaded_dataframes() creates 8 ChromaDB collections via Chroma.from_documents()
7. Ingestion status is persisted to data/.ingestion_status.json (run-once lock)
8. clear_resource_caches() invalidates all @st.cache_resource so new DBs are picked up

6. Quality Control — Self-RAG Implementation

The system implements a Self-RAG quality loop inspired by CRAG (Corrective RAG) and Self-Reflective RAG patterns. Every agent output passes through two quality checkpoints before being returned to the user.

QC Node	Check	Mechanism	Outcome
check_hallucination	Factual Grounding	LLM compares answer against retrieved context + web evidence; returns "Fully Supported", "Partially Supported", or "No Support"	Fully Supported → check_usefulness; else → revise (max 1 revision)
revise	Answer Correction	LLM rewrites the answer using ONLY the verified context and web evidence; maintains markdown table format	Revised answer → back to check_hallucination
check_usefulness	Actionability	LLM evaluates "Is this recommendation useful and actionable for the organizer?" → "Useful" or "Not Useful"	Useful → END; Not Useful + rewrite_count < 1 → rewrite_query
rewrite_query	Query Improvement	LLM writes an improved, more specific retrieval query based on the original failure	New query → re-route to original agent or coordinator

Guardrails: MAX_REVISIONS = 1, MAX_REWRITES = 1 (just for testing, can be increased) to prevent infinite loops. All QC results are tracked in quality_results list and displayed in the Execution Monitor tab.

7. Multi-Agent Coordination

7.1 Coordinator Node

When the LLM planner selects 2 or more agents, control passes to coordinator_node instead of a single subgraph. The Coordinator:

9. Receives the ordered agent list from the router (topologically sorted by dependency handoffs)
10. Iterates through each agent sequentially
11. For each agent, passes full agent_state including shared_context (accumulated from prior agents)
12. Invokes the agent's compiled subgraph via _run_agent_subgraph()
13. Extracts the agent's answer and appends "[AGENT] answer" to shared_context
14. Collects all execution logs, state flows, and quality results
15. After all agents complete, routes to combine_results for synthesis

7.2 Result Synthesis

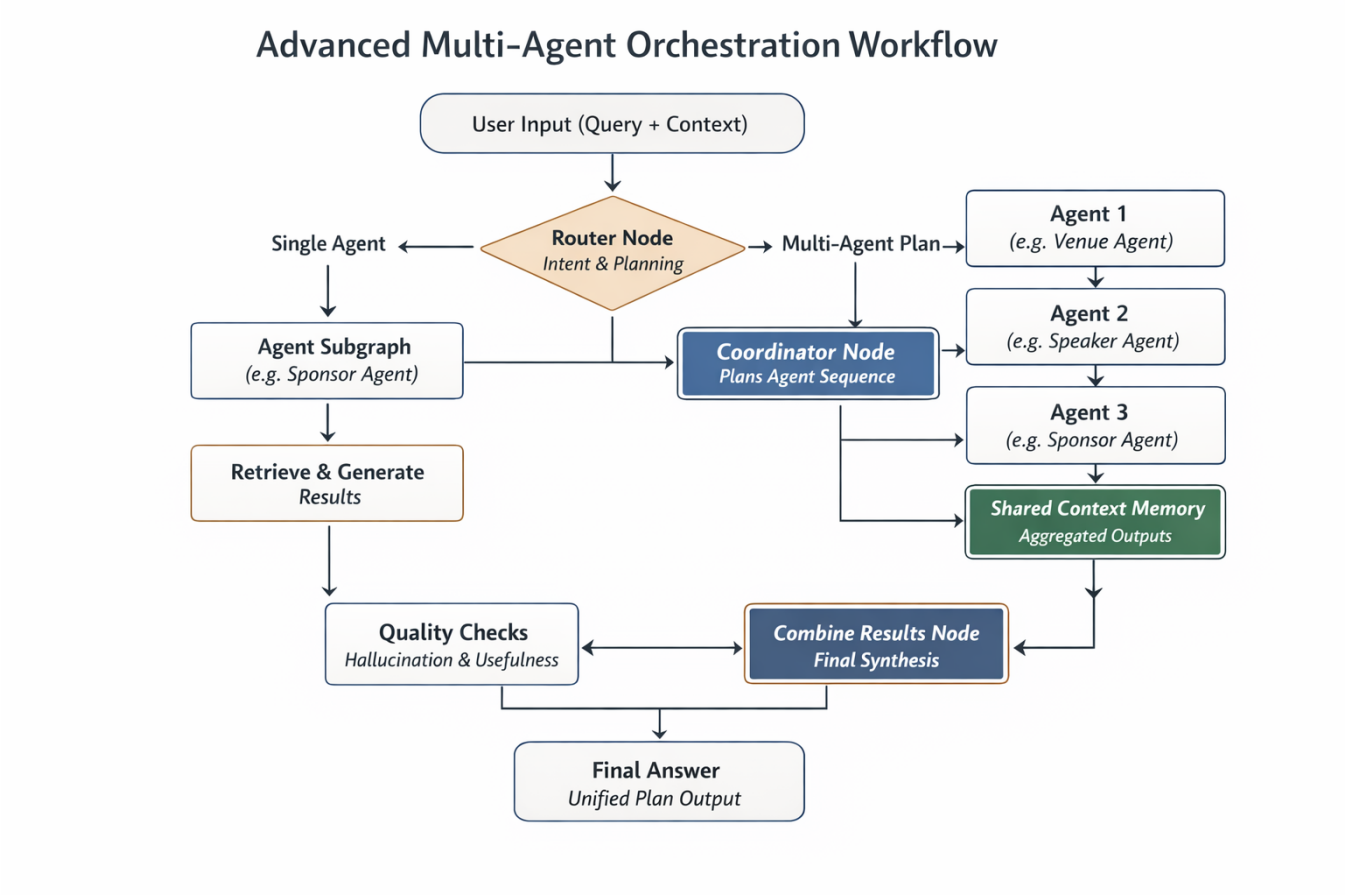
combine_results sends the full agent_outputs dict plus the orchestration plan to the LLM with the instruction: "Create one integrated, actionable response that resolves overlaps between agents, highlights dependencies, and produces next steps." This ensures the multi-agent response reads as a unified plan, not a concatenation.

7.3 Agent Handoffs and Dependency Ordering

The LLM planner can specify handoffs — directed edges between agents — which express that one agent's output should inform another's. The _apply_dependency_order() function runs topological sort (BFS Kahn's algorithm) on the handoff graph to ensure agents run in the correct order. Example: SPONSOR → EMAIL_OUTREACH means sponsor results are available when the email agent runs.

7.4 Context Passing Strategy

shared_context accumulates outputs from each completed agent. When the next agent runs, its context field is set to shared_context, making prior agents' findings available for retrieval enrichment and LLM generation. This creates implicit knowledge transfer across agents without explicit hardcoded dependencies.



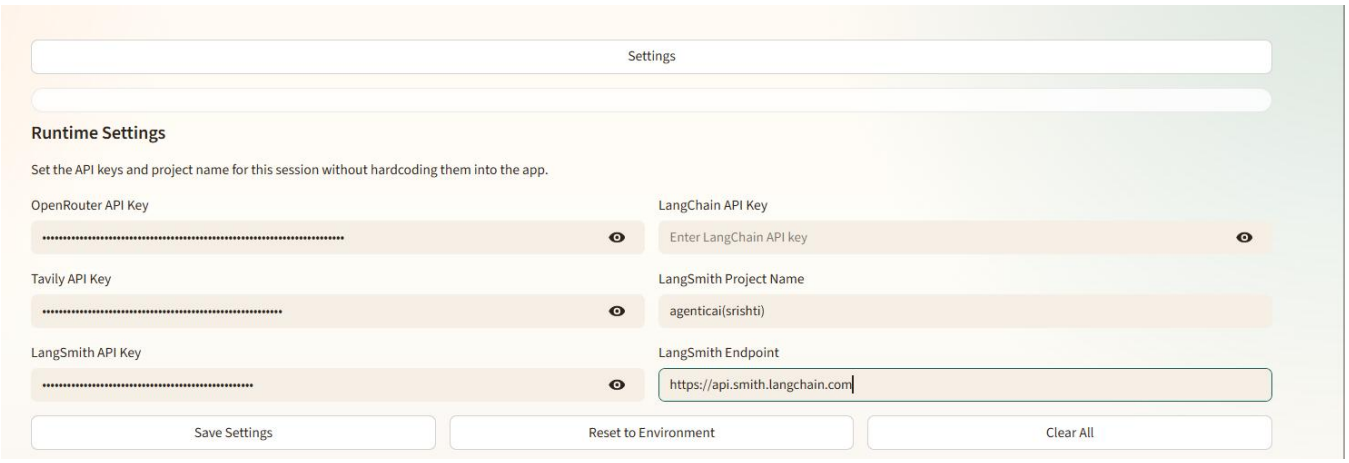
8. User Interface — Streamlit Application

8.1 Control Panel (Sidebar)

The sidebar provides all event configuration inputs:

- Event Request: free-text area for the user query (e.g., "Find sponsors and speakers for an AI conference in Bangalore")
- Category: event category (AI, Web3, SaaS, etc.)
- Location / Country and City: geographic targeting
- Audience Size: numeric input (0–500,000)
- Budget: dropdown (low, medium, high, enterprise)
- Event Topic and Event Name: additional context
- "Run LangGraph Planner" button: triggers full agent execution

Sidebar defaults are auto-populated from the first row of events_v2.csv using get_sidebar_defaults() for a fast demo experience.

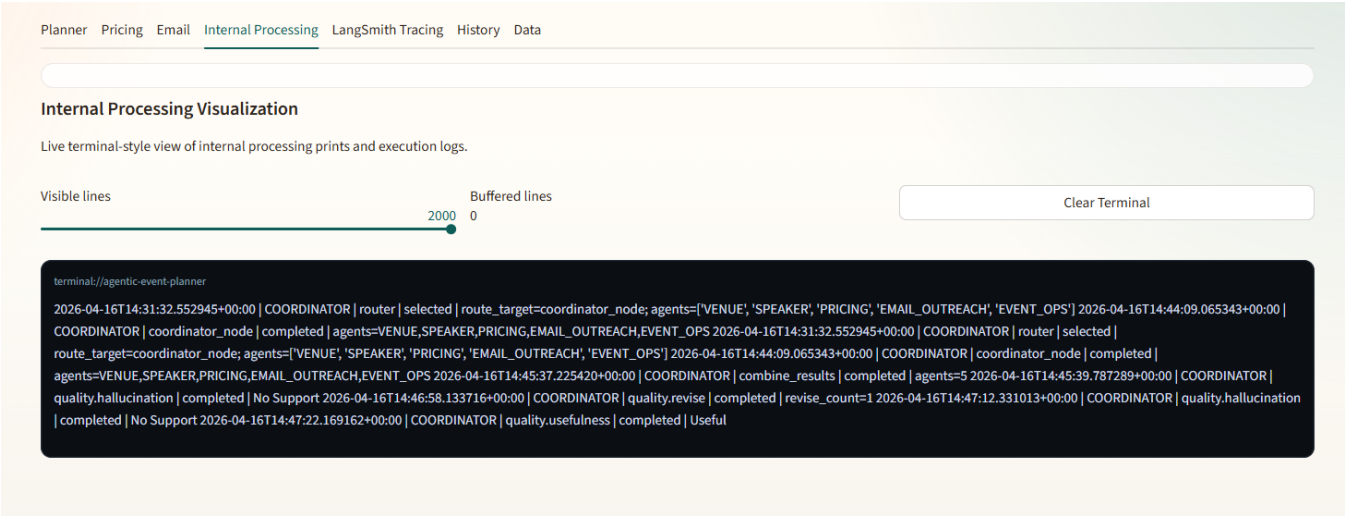


8.2 Seven Application Tabs

Tab	Content
1. Planner	Agent output (markdown tables), request details expander, quality control metrics, Execution Monitor (state flow, execution logs, orchestration plan, agent sequence, quality results)
2. Pricing	"Run Pricing Engine" button; strategy & insights text; Ticket Tier Simulator with sliders; Break-Even Analysis with cost sliders and line chart; Revenue & Conversion Forecasting with what-if sliders and timeseries
3. Email	Contact retrieval query; Load Contacts; contact preview table; Generate Drafts (campaign brief, sender name, tone); per-recipient data_editor with approve checkbox; SMTP credentials; Send Approved / Send All buttons with status
4. Internal Processing	Live terminal-style dark UI showing all Python print() statements captured via custom print() override; buffered up to 4,000 lines; configurable visible lines; clear button
5. LangSmith Tracing	Project selector; limit slider; failed-only filter; stale/latency thresholds; status/type distribution bar charts; latency & token trend line chart; full trace table; per-run JSON detail viewer
6. History	SQLite per-session history (session_id, agent, route, query, output); row limit slider; download CSV button
7. Data	File uploader for all 11 CSVs; preview with row counts; one-time ingestion with lock; dataset viewer with column info; download CSV buttons

8.3 Terminal Output Capture

A custom print() function override intercepts all stdout output and appends each line to st.session_state["terminal_output_lines"]. This is rendered in the Internal Processing tab as a dark terminal display using HTML/CSS with escape. Buffer max is 4,000 lines (configurable via TERMINAL_BUFFER_MAX_LINES env var).



9. Observability & Monitoring

9.1 Execution Monitor

Every agent node calls build_monitor_update() which appends structured entries to three lists: logs (text), execution_logs (dicts with timestamp/agent/node/status/details), and state_flow (node names). These are surfaced in Tab 1 as a state flow code block, a dataframe, and quality results.

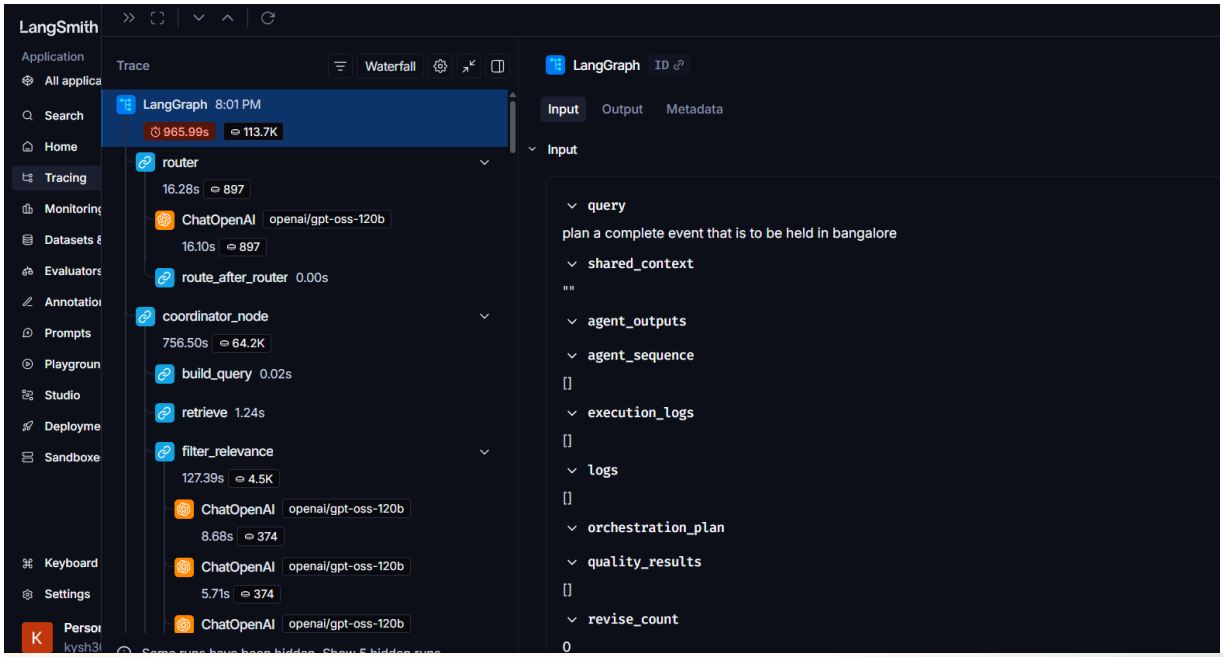
9.2 LangSmith Integration

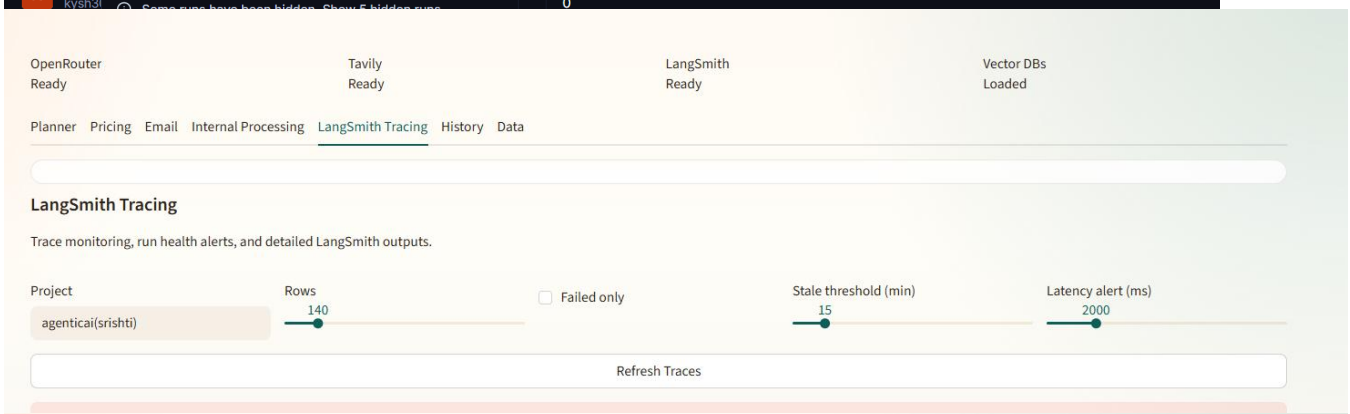
The platform integrates with LangSmith for production-grade LLM observability:

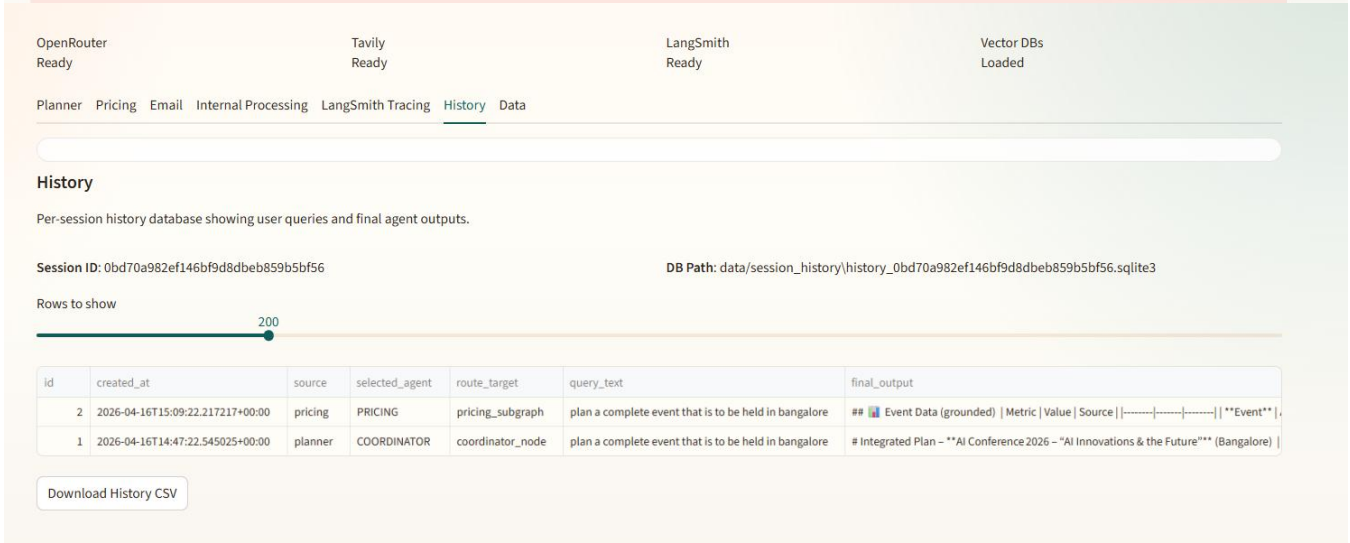
- fetch_langsmith_traces() calls the LangSmith Python SDK client.list_runs() for the configured project
- Displays: total runs, failed count, error rate %, avg latency ms, total tokens, last trace age
- Alerts on: failed traces, latency above threshold, stale traces
- Visualizations: status distribution bar chart, run type distribution, latency/token trend line chart
- Per-run JSON detail viewer for deep inspection
- Configuration: LANGSMITH_API_KEY, LANGCHAIN_PROJECT, LANGCHAIN_ENDPOINT environment variables

9.3 Session History

Every planner and pricing execution is logged to a per-session SQLite database at data/session_history/history_{session_id}.sqlite3. Schema: id, created_at, query_text, final_output, selected_agent, route_target, source. Entries are downloadable as CSV from Tab 6.


LangSmith
Application
All applica
Search
Home
Tracing
Monitoring
Datasets &
Evaluators
Annotatio
Prompts
Playgroun
Studio
Deployme
Sandboxe
Keyboard
Settings
Trace
Waterfall
LangGraph 8:01 PM
965.99s
113.7K
router
16.28s 897
ChatOpenAI openai/gpt-oss-120b
16.10s 897
route_after_router 0.00s
coordinator_node
756.50s 64.2K
build_query 0.02s
retrieve 1.24s
filter_relevance
127.39s 4.5K
ChatOpenAI openai/gpt-oss-120b
8.68s 374
ChatOpenAI openai/gpt-oss-120b
5.71s 374
ChatOpenAI openai/gpt-oss-120b
LangGraph ID
Input Output Metadata
Input
query
plan a complete event that is to be held in bangalore
shared_context
""
agent_outputs
agent_sequence
[]
execution_logs
[]
logs
[]
orchestration_plan
quality_results
[]
revise_count
0


OpenRouter
Ready
Tavily
Ready
LangSmith
Ready
Vector DBs
Loaded
Planner Pricing Email Internal Processing LangSmith Tracing History Data
LangSmith Tracing
Trace monitoring, run health alerts, and detailed LangSmith outputs.
Project
agenticai(srishti)
Rows
140
Failed only
Stale threshold (min)
15
Latency alert (ms)
2000
Refresh Traces


OpenRouter
Ready
Tavily
Ready
LangSmith
Ready
Vector DBs
Loaded
Planner Pricing Email Internal Processing LangSmith Tracing History Data
History
Per-session history database showing user queries and final agent outputs.
Session ID: 0bd70a982ef146bf9d8dbeb859b5bf56
DB Path: data/session_history\history_0bd70a982ef146bf9d8dbeb859b5bf56.sqlite3
Rows to show
200
id | created_at | source | selected_agent | route_target | query_text | final_output
2 | 2026-04-16T15:09:22.217217+00:00 | pricing | PRICING | pricing_subgraph | plan a complete event that is to be held in bangalore | ## Event Data (grounded) | Metric | Value | Source | |---|---|---|
1 | 2026-04-16T14:47:22.545025+00:00 | planner | COORDINATOR | coordinator_node | plan a complete event that is to be held in bangalore | # Integrated Plan – **AI Conference 2026 – "AI Innovations & the Future"** (Bangalore) |
Download History CSV

10. Technology Stack

Component	Technology	Version / Notes
Orchestration	LangGraph (StateGraph)	Multi-agent state machine with subgraphs and conditional edges
LLM	OpenRouter → GPT-4 / openai/gpt-oss-120b	Via LangChain ChatOpenAI with OpenRouter base_url
Embeddings	HuggingFace sentence-transformers/all-MiniLM-L6-v2	Cached @st.cache_resource; 384-dim vectors
Vector Store	ChromaDB	8 persisted collections on disk
RAG Framework	LangChain	Document, HuggingFaceEmbeddings, Chroma, similarity_search
ML Models	scikit-learn RandomForestRegressor	Attendance model (300 trees), Conversion model (250 trees)
Web Search	Tavily	Real-time enrichment per agent candidate (advanced search depth)
UI Framework	Streamlit	7-tab layout, sidebar, data_editor, charts, file_uploader
Email	smtplib (SMTP/Gmail)	STARTTLS, app password, MIMEText
Session Storage	SQLite (sqlite3)	Per-session history database
Tracing	LangSmith SDK	Run listing, metrics, token/latency analytics
Data Processing	pandas, numpy	CSV ingestion, preprocessing, feature engineering
ML Pipeline	sklearn Pipeline, ColumnTransformer	Imputation, OneHotEncoding, RF model in single pipeline object
Environment	python-dotenv	OPENROUTER_API_KEY, TAVILY_API_KEY, LANGSMITH_API_KEY, etc.

11. Complete End-to-End Workflow

Step 1: Data Preparation (One-Time Setup)

16. Prepare 11 CSV datasets covering events, sponsors, speakers, exhibitors, venues, communities, sessions, rooms, time_slots, pricing_tiers, contacts
17. Upload CSVs via Tab 7 → "Ingest Now" button
18. System builds 8 ChromaDB vector stores from documents and persists to disk
19. PricingEngine loads data, preprocesses, trains RandomForest models on first pricing query

Step 2: Event Configuration

20. User fills sidebar: category (e.g., AI), location (India), city (Bangalore), audience_size (5000), budget (high), event_topic (Artificial Intelligence)
21. User writes a natural language query: e.g., "Find the best sponsors and speakers for our AI conference in Bangalore with 5000 attendees"

Step 3: LangGraph Planner Execution

22. User clicks "Run LangGraph Planner"
23. router_node: LLM planner parses query, returns {"agents": ["SPONSOR", "SPEAKER"], "handoffs": [{"from": "SPONSOR", "to": "SPEAKER", "reason": "speaker outreach uses sponsor context"}], "reasoning": "..."}
24. required_agents has 2 → routes to coordinator_node
25. coordinator_node: runs SPONSOR subgraph first
26. Sponsor pipeline: build_query("AI conference sponsors in India audience 5000 high budget") → retrieve_from_db(k=12) → filter_relevance (LLM YES/NO) → enrich_web (Tavily per company) → generate_sponsor (LLM → markdown tables)
27. Coordinator appends sponsor answer to shared_context
28. coordinator_node: runs SPEAKER subgraph with shared_context including sponsor findings
29. Speaker pipeline: similar flow → speaker prioritization + agenda mapping tables
30. Coordinator sends all outputs to combine_results
31. combine_results: LLM synthesizes into one unified plan with cross-agent insights
32. check_hallucination: verifies combined answer against all contexts → "Fully Supported"
33. check_usefulness: confirms answer is actionable → "Useful" → END
34. UI renders final_answer in Tab 1 with state flow and quality metrics

Step 4: Pricing Analysis

35. User switches to Tab 2, clicks "Run Pricing Engine"
36. Graph invoked with selected_agent="PRICING"
37. Pricing subgraph: builds query → retrieves from pricing_vector_db → filters relevance → Tavily market research → generate_pricing calls PricingEngine.run()

38. PricingEngine: scans 40 price points, predicts attendance + conversion, selects revenue-maximizing price, generates 3 tiers with seat allocation
39. LLM generates strategy narrative + 3 insights
40. Tab 2 shows: strategy text + Ticket Tier Simulator + Break-Even Analysis + Revenue Forecasting

Step 5: Email Outreach

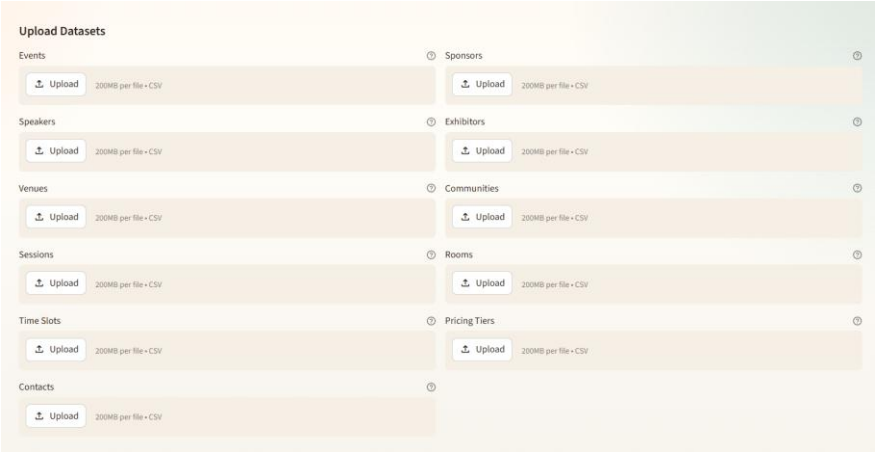
41. User switches to Tab 3, enters contact retrieval query, loads contacts
42. System retrieves top-k contacts from contact_vector_db by semantic similarity
43. User enters campaign brief (pre-filled from planner output), sender name, tone
44. Generate Drafts: per-contact LLM call → {"subject": ..., "body": ...}
45. User reviews and edits drafts in data_editor, checks approve boxes
46. User enters Gmail + app password, clicks "Send Approved"
47. System sends via smtplib.SMTP with STARTTLS; updates status per row

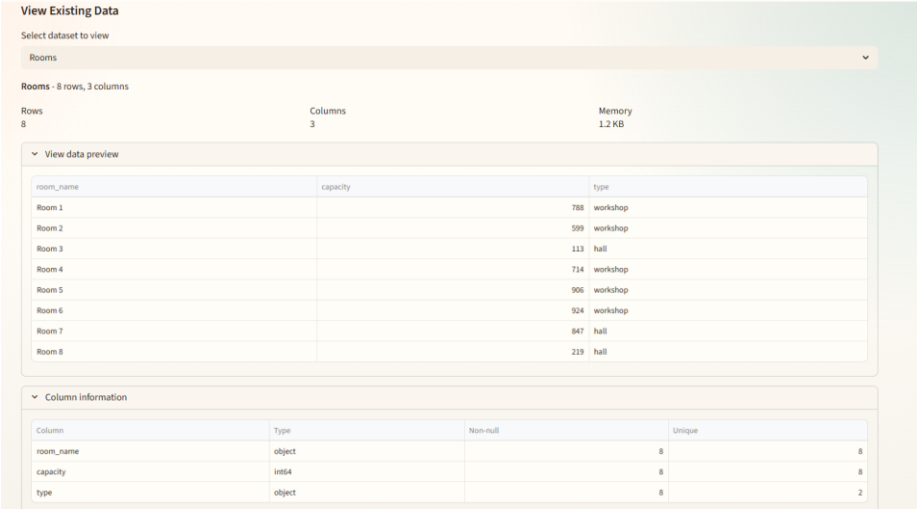
Step 6: Monitoring & Tracing

48. Tab 4 (Internal Processing): real-time terminal showing all agent execution steps
49. Tab 5 (LangSmith): token usage, latency, error rates, per-run JSON inspection
50. Tab 6 (History): full query/output history downloadable as CSV

12. Data Sources & Extraction

Dataset	Source / Method	Notes
events_v2.csv	Curated and generated; patterns based on real conference data from Eventbrite, Cvent, eventlocations.com, public conference websites	Covers 2025-2026; 4 geographies; multiple categories
sponsors_v2.csv	Based on known tech sponsors (Meta, Google, Microsoft, NVIDIA, AWS, Intel, IBM, etc.) from public conference sponsor lists	Normalized tier structure (Gold, Silver, Bronze)
speakers_v2.csv	Based on known AI/Tech thought leaders (Sundar Pichai, Sam Altman, Fei-Fei Li, Andrew Ng, etc.) from public conference programs	Mapped to events and topics
exhibitors.csv	Based on typical tech conference exhibitor profiles; industry clusters (startup, enterprise, tools)	Booth type normalized
venues.csv	Based on major conference venues in target cities; capacity and cost estimates from venue websites	past_events field links to event names for ML join
communities.csv	Based on publicly listed Discord servers, Slack workspaces, LinkedIn groups, Meetup groups for tech/AI communities	Members and activity estimated from public data
sessions.csv, rooms.csv, time_slots.csv	Synthetically generated following real conference scheduling patterns (keynotes, workshops, panels, breaks)	Structured for conflict detection demo
pricing_tiers.csv	Based on real conference pricing from Eventbrite, conference websites; tiers: early_bird, regular, vip, group	Includes historical conversion_rate data
contacts.csv	Synthetically generated contact list of sponsor/speaker/community/venue contacts with company, role, email, LinkedIn fields	Designed for outreach demo





13. Key Design Decisions

Decision	Why
LangGraph over custom orchestration	State machine provides deterministic routing, built-in conditional edges, compiled subgraphs that run independently, and native LangSmith tracing integration
Subgraph-per-agent architecture	Each agent is a fully compiled LangGraph subgraph — independently testable, replaceable without affecting other agents, and reusable in any combination
Shared GraphState TypedDict	Single state object enables clean context passing between agents; coordinator can accumulate shared_context without complex message passing
8 separate ChromaDB stores (not 1)	Agent-specific stores keep retrieval focused; prevents sponsor queries from polluting speaker results; enables per-agent tuning of k and relevance thresholds
Tavily enrichment per candidate (not per query)	More targeted web profiles per entity (sponsor company, speaker name, venue name) rather than generic category search; enables signal_score computation
PricingEngine as separate class	Isolates ML complexity from agent orchestration; reusable and testable independently; can be upgraded to XGBoost or deep learning without touching agents
One-time ingestion lock	Prevents accidental re-ingestion on every page reload; ingestion_status.json persists across Streamlit sessions; user can trigger manually from Data tab
LLM relevance filter (not just vector similarity)	Pure similarity search returns semantically close but contextually irrelevant documents; LLM YES/NO filter adds event-context-aware relevance judgment
Self-RAG QC loop (hallucination + usefulness)	Two-stage quality gate catches both factual errors (hallucination) and practical failures (not actionable); max 1 revision and 1 rewrite prevent infinite loops
SQLite for session history	Lightweight, file-based, no server required; per-session isolation; trivially downloadable as CSV

14. Bonus Features Implemented

Beyond the core PS1 requirements, the following bonus features were implemented:

Bonus Feature	Where	Details
Revenue Projection	Tab 2 — Pricing Studio	Tier-level revenue calculation with promo discount; cumulative forecast timeseries
Ticket Tier Simulation	Tab 2 — Simulator	Interactive sliders for price/seats/conversion per tier; live bar chart; gross vs. promo revenue
Break-Even Analysis	Tab 2 — Break-Even	Cost sliders (venue, speaker, ops, marketing); days-to-breakeven; feasibility flag; revenue vs. fixed-cost curve
What-If Scenario Forecasting	Tab 2 — Forecasting	Price shift, conversion shift, demand shift, promo discount sliders; per-tier impact analysis; 26-week horizon with growth trends
Email Outreach Agent	Tab 3 — Email	Full SMTP pipeline with personalized LLM drafts, per-recipient editing, approval flow, and send status
Live Terminal Visualization	Tab 4 — Internal Processing	Dark terminal showing real-time agent execution prints; buffer management; clear button
LangSmith Trace Monitoring	Tab 5 — LangSmith	Error rate, latency alerts, stale trace detection, per-run JSON inspection, token analytics
SQLite Session History	Tab 6 — History	Per-session query/output logging with CSV export
Domain-Agnostic Architecture	Architecture	Agent category is parameterized from user_input; swapping category to "Music Festival" or "Sporting Event" produces domain-appropriate outputs
Multi-Agent Collaboration	Coordinator + Synthesizer	LLM plans agent sequences with dependency handoffs; combine_results produces integrated cross-agent plan

15. Summary & Conclusion

The Agentic Event Planner is a complete, production-ready multi-agent AI system that addresses every requirement of the HighPrep PS1 problem statement. The system brings together:

- 8 specialized agents (Sponsor, Speaker, Exhibitor, Venue, Pricing, Community, Event Ops, Email Outreach)
- LangGraph state machine orchestration with LLM-powered planning, dependency ordering, and result synthesis
- 11 clean CSV datasets covering 2025-2026 events across 4 geographies and multiple categories
- 8 ChromaDB vector stores with HuggingFace embeddings for semantic RAG retrieval
- Tavily real-time web enrichment for candidate validation beyond historical data
- ML PricingEngine with RandomForest models for attendance prediction, conversion prediction, and price optimization
- Full financial toolset: tier simulator, break-even analysis, what-if revenue/conversion forecasting
- SMTP email outreach with personalized LLM-generated drafts and approval workflow
- Self-RAG quality control with hallucination detection, revision, and usefulness check
- LangSmith observability with trace monitoring, error alerts, and token analytics
- Streamlit UI with 7 functional tabs and live execution visualization

The architecture is explicitly designed to be domain-agnostic — swapping the category from "AI Conference" to "Music Festival" or "Sporting Event" immediately adapts all agent queries, ChromaDB retrieval, and LLM generation to the new domain, satisfying the PS1 extra-credit requirement for multi-domain extensibility.

— End of Report —